

Differences Between Finalize and Dispose in C#

In this article, I will explain the **Differences Between Finalize and Dispose in C#** with Example. Please read our previous article discussing [Garbage Collection in .NET Framework](#). In C#, Finalize and Dispose are both methods used to release resources, but they serve different purposes and are used in different scenarios:

Finalize Method in C#:

- **Purpose:** The Finalize method is used for cleanup operations before an object is garbage collected. It's typically overridden to release unmanaged resources that the object holds. The garbage collector calls the Finalize method automatically.
- **Control:** You do not call Finalize directly. It's invoked by the garbage collector.
- **Non-deterministic:** The exact time when Finalize is called is non-deterministic, depending on the garbage collector's schedule.
- **Inheritance:** The Finalize method is inherited from the Object class. It should always call the Finalize method of its base class if overridden to ensure that all resources are released properly.

Dispose Method:

- **Purpose:** The Dispose method is part of the IDisposable interface and is implemented to release both managed and unmanaged resources deterministically.
- **Control:** Unlike Finalize, Dispose is called explicitly in your code, usually when you are done using an object. This allows for the immediate freeing of resources.
- **Deterministic:** Dispose provides a deterministic way to release resources, meaning you know exactly when the resources are released.
- **Pattern:** When implementing Dispose, it's common to follow the dispose pattern, which includes a finalizer call (GC.SuppressFinalize(this)) to prevent the garbage collector from calling Finalize if Dispose has already been called.

Example to Understand finalize and dispose in C#

Let us see an example to understand the use of Finalize and Dispose methods in resource management. In this example, we'll create a simple class named ResourceHolder that simulates the management of an unmanaged resource. The following example is self-explained, so please go through the comment line for better understanding.

```
using System;
namespace GarbageCollectionDemo
{
    public class ResourceHolder : IDisposable
    {
        // To track whether Dispose has been called.
        private bool _disposed = false;
        // Constructor
        public ResourceHolder()
        {
            // Allocate or initialize an unmanaged resource.
            Console.WriteLine("Unmanaged resource allocated.");
        }
    }
}
```

```

}
// Implementing Dispose method from IDisposable interface
public void Dispose()
{
    Dispose(true);
    GC.SuppressFinalize(this); // Prevent finalizer from being called.
}
protected virtual void Dispose(bool disposing)
{
    if (!_disposed)
    {
        if (disposing)
        {
            // Free any other managed objects here.
            Console.WriteLine("Free other managed objects");
        }
        // Free unmanaged resources here.
        Console.WriteLine("Unmanaged resource released.");
        _disposed = true;
    }
}
// Finalizer is nothing but the destructor
~ResourceHolder()
{
    Dispose(false);
    Console.WriteLine("Finalizer called.");
}
}
//How to Use the above class
class Program
{
    static void Main(string[] args)
    {
        // Using the ResourceHolder with using statement
        using (var resourceHolder = new ResourceHolder())
        {
            // Use the resource...
        } // Dispose is called automatically when exiting the using block.
        // If not using 'using', dispose should be called manually.
        var anotherResourceHolder = new ResourceHolder();
        // Use the resource...
        anotherResourceHolder.Dispose();
        // Without calling Dispose, finalizer will be called by GC at some
        var finalResourceHolder = new ResourceHolder();
        // Use the resource...
        Console.ReadKey();
    }
}
}

```

Output:

```
Unmanaged resource allocated.  
Free other managed objects  
Unmanaged resource released.  
Unmanaged resource allocated.  
Free other managed objects  
Unmanaged resource released.  
Unmanaged resource allocated.
```

Explanation:

ResourceHolder Class:

- The ResourceHolder class simulates the management of an unmanaged resource.
- The Dispose method is implemented from the IDisposable interface. It's called to release resources deterministically.
- The protected Dispose(bool disposing) method performs the actual resource cleanup. The disposing parameter indicates whether the method is being called from the Dispose method or from the finalizer.
- The finalizer (~ResourceHolder) is called by the garbage collector if the object is not disposed of properly. It calls Dispose(false).

Using the Class:

- The using statement ensures that Dispose is called automatically for resourceHolder.
- If not using using, Dispose should be called manually as shown with anotherResourceHolder.
- finalResourceHolder demonstrates a scenario where Dispose is not called, hence the finalizer will eventually be invoked by the garbage collector.

The example above illustrates the proper implementation and usage of both Dispose and Finalize for resource management in C#.

Key Differences Between Finalize and Dispose in C#:

- **Timing:** Finalize is called by the garbage collector in a non-deterministic manner, while Dispose is called explicitly at a known point in the program.
- **Resources:** Finalize is typically used for unmanaged resources, whereas Dispose can be used for both managed and unmanaged resources.
- **Control:** Dispose gives you more control over resource management compared to Finalize.

Best Practices:

- Implement Dispose to allow deterministic cleanup of resources.
- Use a finalizer (Finalize method) only for cleaning up unmanaged resources that are not wrapped in a safe handle and when there's no guarantee that Dispose will be called.

- In the Dispose method, call GC.SuppressFinalize to prevent the garbage collector from calling Finalize, if it has already been disposed of.
- Follow the dispose pattern, especially if your class owns unmanaged resources.

In the next article, I will discuss **Access Specifiers in C#** with Examples. In this article, I explain the **Differences Between Finalize and Dispose in C#** with Example. I hope you enjoy this Differences Between Finalize and Dispose in C# with Example article.